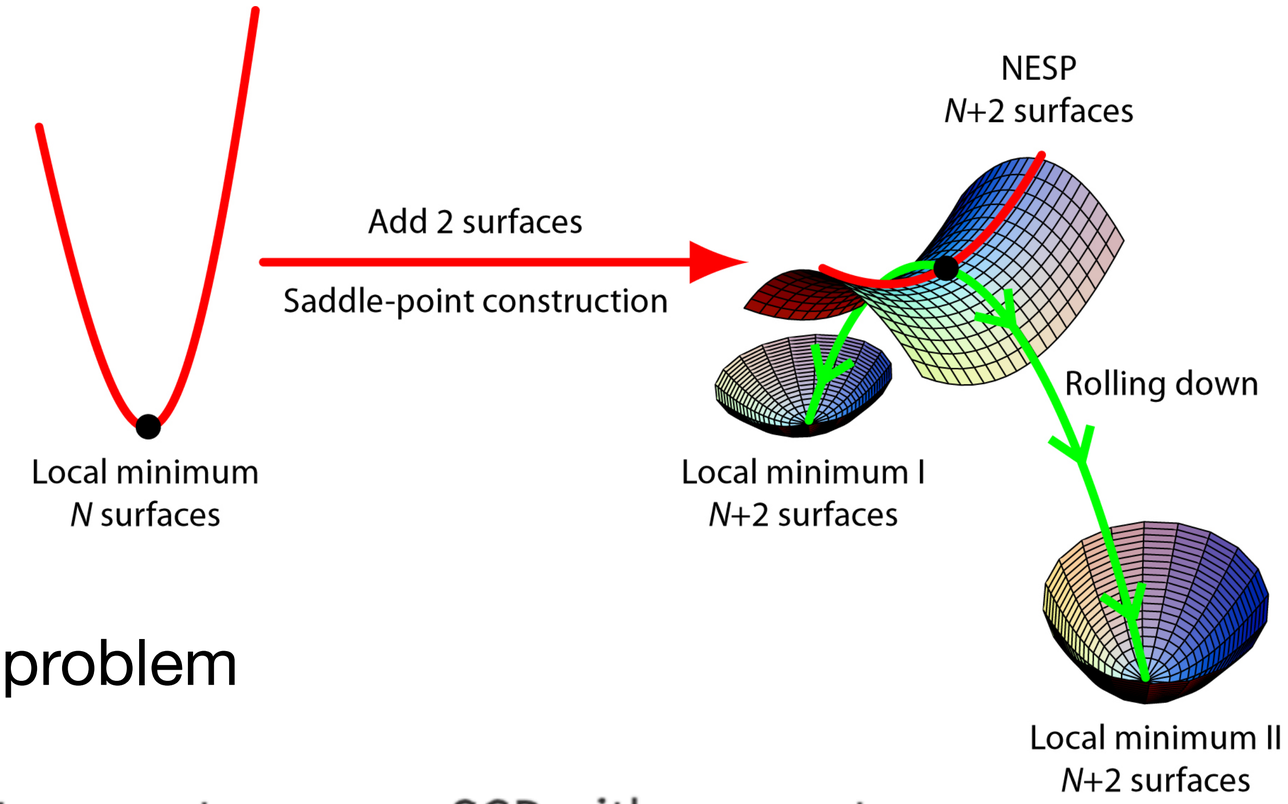
Week4 Optimizer

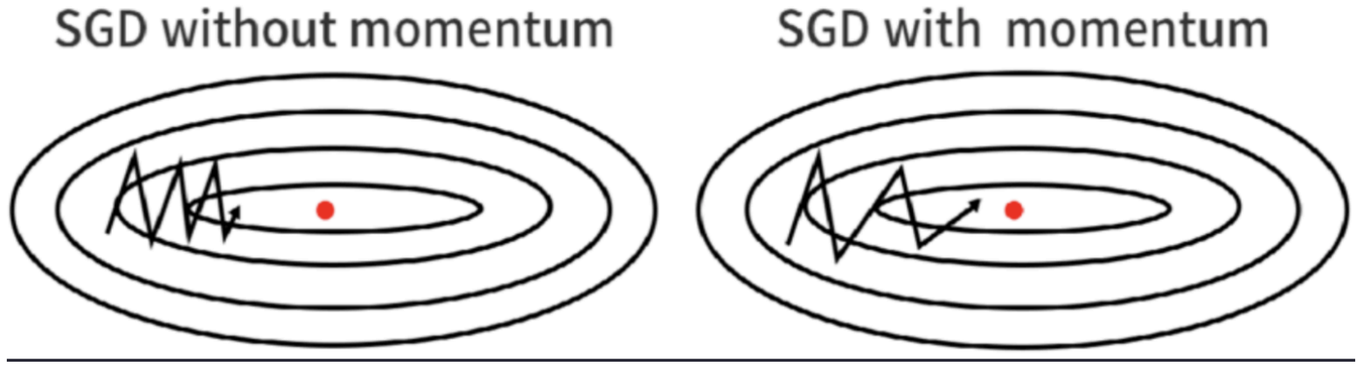
Tutor:
Email:
Tutorial:
Code: https://github.com/Jinxu-Lin/COMP5329

Optimizer

Momentum

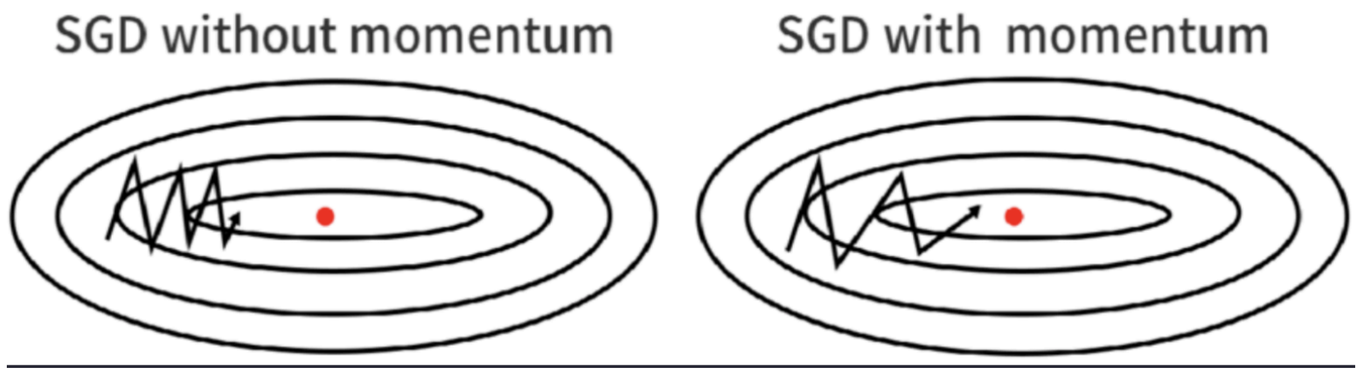
- Benefit:
  - Avoid saddle point
  - Learning rate selection problem





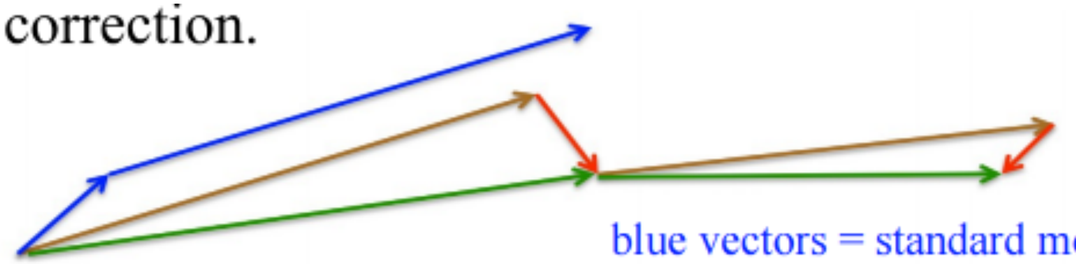
Standard Momentum

- 1. Initialize v_t as 0
- 2. Update momentum: $v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$
- 3. Update parameters: $\theta_t = \theta_{t-1} - v_t$



Nesterov accelerated gradient

- 1. Initialize v_t as 0
- 2. Estimate the next position using history gradients (**make big jump**)
  - $\theta_t^e = \theta_{t-1} - \gamma v_{t-1}$
- 3. Update momentum based on estimated position (**correction**)
  - $v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta_t^e)$ correction.
- 4. Update parameters
  - $\theta_t = \theta_{t-1} - v_t$



https://isaacchanghau.github.io/2017/05/29/parameter-update-methods/

blue vectors = standard momentum
brown vector = jump, red vector = correction,
green vector = accumulated gradient ,

Adaptive Learning Rate Methods

- Benefit: Hemogeneous learning rate problem
  - AdaGrad
  - RMSProp
  - Adam

AdaGrad (Adaptive Gradient Algorithm)

- $\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} * g_{t,i}$, where
  - $g_{t,i} = (\nabla_\theta J(\theta_t))_i$ and $G_{t,ii} = \sum_{i=1}^{t} g_{t,i}^2$
- Objective:
  - Perform larger updates for infrequent updated parameters
  - Smaller updates for frequent updated parameters

RMSProp (Root Mean Square Propagation)

- AdaGrad Problems:
  - Learning rate decay helps to increase the learning rate for infrequent parameters
  - However, it also reduce the ability of learning for frequent parameters
- => The idea is to also do decay for the adaptive parameters

RMSProp

- $\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} * g_{t,i}$, where
  - $g_{t,i} = (\nabla_\theta J(\theta_t))_i$ and $G_{t,ii} = \gamma G_{t-1,ii} + (1-\gamma) g_{t,i}^2$
- Notice: Similar to AdaGrad, just to modify the $G_{t,ii}$ by EMA.
- RMSProp = AdaGrad + momentum

Adam

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
- Bias-correction
  - $\hat{v}_t = \dfrac{v_t}{1 - \beta_2^t}, \quad \hat{m}_t = \dfrac{m_t}{1 - \beta_1^t}$
- $\theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\hat{v} + \epsilon}} \hat{m}_t$

Exam-style Questions

Code

Codes

- ./materials/Week4_Optimizer/Week4_Optimizer.ipynb
- ./ResNet/Optimizers/sgd.py
- ./ResNet/Optimizers/adam.py
- ./ResNet/train.py: line206-207
- ./ResNet/train_utils.py: line47